

Question Bank — Week 13

Hash Tables (Lecture 13) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice	2
Part B — Short answer and essay	5
Part C — Trace the code	6
Part D — Table state: draw every step	6
Part E — Complexity analysis	7
Part F — Find and fix the bug	7
Part G — Write the code	8

Answers are in a separate file: [week13-answers.md](#). Levels: **[what]** recall · **[how]** apply · **[why]** explain and justify. ChainingHashMap and OpenAddressingHashMap are the classes of `dsa/hashmap.py`. Both start at **capacity 8** unless a question says otherwise, and double when they resize. The index of a key is `hash(key) % capacity`, and every key in these questions is an **integer**, so `hash(k)` is `k`.

- ChainingHashMap (max_load 0.75): a new key is pushed on the **front** of its chain; after the insert, if `size / capacity > 0.75`, the table resizes to twice the capacity, re-linking every entry onto the front of its new chain, old buckets in order 0, 1, 2, ...
- OpenAddressingHashMap (max_load 0.66): linear probing, `h`, `h+1`, `h+2`, ... modulo the capacity; delete leaves a **tombstone** (`!`). **Before** an insert, if `(size + tombstones + 1) / capacity > 0.66`, the table is rebuilt without tombstones — at twice the capacity if `(size + 1) / capacity > 0.66`, otherwise at the same capacity. A put that passes a tombstone remembers the first one, probes on to None, and inserts at that tombstone.

A **probe** is one slot examined. Slots are written left to right, 0 to 7, with `_` for a never-used slot.

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W13-M01 – W13-M22
B	Short answer and essay	W13-E1 – W13-E5

Part	Type	Questions
C	Trace the code	W13-T1 – W13-T5
D	Table state — draw every step	W13-S1 – W13-S3
E	Complexity analysis	W13-K1 – W13-K3
F	Find and fix the bug	W13-B1 – W13-B4
G	Write the code — checked by pytest	W13-C1 – W13-C5

Part A — Multiple choice

W13-M01 [what] A hash table finds the slot of a key by computing:

- **a)** `key // capacity`
- **b)** `hash(key) % capacity`
- **c)** a binary search over the slots
- **d)** `hash(key) * capacity`

W13-M02 [how] In a table of capacity 8, the index of the key 45 is:

- **a)** 5
- **b)** 4
- **c)** 3
- **d)** 45

W13-M03 [what] The one rule every hash function **must** obey is:

- **a)** different keys have different hashes
- **b)** the hash is a prime number
- **c)** equal keys have equal hashes
- **d)** the hash is smaller than the capacity

W13-M04 [why] `python -c "print(hash('abc'))"` prints a different number each time you run it because:

- **a)** it is a bug in Python
- **b)** the hash of a string is its address in memory
- **c)** the capacity of the table changed
- **d)** string hashing uses a random secret key chosen at start-up, on purpose

W13-M05 [how] Which of these is the same in every run of Python?

- **a)** `hash(12345)`
- **b)** `hash("12345")`
- **c)** `hash(b"12345")`
- **d)** the order in which a set of strings is printed

W13-M06 [why] `m.put([1, 2], "x")` raises `TypeError: unhashable type: 'list'` because:

- **a)** a list can change after it is stored, and then it would be in the wrong bucket
- **b)** lists are too long to hash
- **c)** put only accepts integers
- **d)** the table is full

W13-M07 [how] 23 keys are placed in 365 slots at random. The probability that at least two share a slot is about:

- **a)** 6%
- **b)** 50%
- **c)** 23/365
- **d)** 100%

W13-M08 [what] The load factor of a hash table is:

- **a)** capacity / size
- **b)** the number of collisions so far
- **c)** size / capacity
- **d)** the length of the longest chain

W13-M09 [how] A ChainingHashMap () receives put for 5, 13 and 21. Bucket 5 is:

- **a)** $5 \rightarrow 13 \rightarrow 21$
- **b)** $21 \rightarrow 13 \rightarrow 5$
- **c)** $13 \rightarrow 5 \rightarrow 21$
- **d)** 5 only; 13 and 21 are in other buckets

W13-M10 [how] In the table of W13-M09, get (29) compares 29 with how many entries before raising KeyError?

- **a)** 1
- **b)** 0
- **c)** 3
- **d)** 4

W13-M11 [how] A ChainingHashMap () receives distinct keys one at a time. The first resize happens during the put of the:

- **a)** 6th key
- **b)** 8th key
- **c)** 4th key
- **d)** 7th key

W13-M12 [why] _resize cannot copy old[i] into new[i] because:

- **a)** an Array cannot be copied
- **b)** the index depends on the capacity, so keys would sit where get no longer looks
- **c)** the chains would come out reversed
- **d)** the load factor would be wrong

W13-M13 [how] An OpenAddressingHashMap () receives put for 6, 14 and 22. Key 22 is in slot:

- **a)** 0
- **b)** 6
- **c)** 7
- **d)** it is not stored: slot 6 is taken

W13-M14 [how] In the table of W13-M13, `get(30, None)` makes how many probes?

- **a)** 1
- **b)** 3
- **c)** 5
- **d)** 4

W13-M15 [why] In open addressing, `delete` leaves a tombstone instead of `None` because:

- **a)** a tombstone uses less memory
- **b)** writing `None` is slower
- **c)** `None` would cut the probe sequences that pass through the slot, and keys beyond it would become unreachable
- **d)** an `Array` slot cannot hold `None`

W13-M16 [how] `get` meets a tombstone while probing. It:

- **a)** keeps probing
- **b)** stops: the key is not in the table
- **c)** returns the tombstone
- **d)** raises `KeyError`

W13-M17 [why] `put` counts tombstones in its load check because:

- **a)** tombstones are keys
- **b)** otherwise tombstones can take every `None`, and a probe that stops only at `None` scans the whole table — or never stops
- **c)** `len()` would be wrong otherwise
- **d)** Python requires it

W13-M18 [what] Primary clustering is:

- **a)** keys stored in sorted groups
- **b)** chains growing long in a chaining table
- **c)** runs of occupied slots in linear probing, which grow faster the longer they are
- **d)** several hash tables sharing one `Array`

W13-M19 [how] For linear probing with a good hash, Knuth's formula for an **unsuccessful** search is $\frac{1}{2}(1 + 1/(1 - \alpha)^2)$. At $\alpha = 0.5$ it gives:

- **a)** 1.5
- **b)** 2.5
- **c)** 4
- **d)** 50.5

W13-M20 [why] The keys 0, 1024, 2048, 3072, ... (n of them) are put into a Chaining-HashMap, whose capacity is always a power of two. While the capacity is at most 1,024, one

get costs:

- **a)** $O(1)$ — the table resizes
- **b)** $O(\log n)$
- **c)** $O(n \log n)$
- **d)** $O(n)$ — every key is in bucket 0

W13-M21 [what] Python's dict is:

- **a)** open addressing, with a probe sequence that mixes in the high bits of the hash; it grows at about two-thirds full
- **b)** chaining, with a linked list in every bucket
- **c)** a balanced binary search tree
- **d)** a sorted array searched with bisect

W13-M22 [why] Which question does a sorted array answer in $O(\log n)$ but a hash table cannot answer faster than $O(n)$?

- **a)** is key k present?
 - **b)** what is the value stored for key k ?
 - **c)** add a new key k
 - **d)** how many keys lie between a and b ?
-

Part B — Short answer and essay

W13-E1 [why] (4 marks) Explain how a hash table finds a key in $O(1)$ time on average. State the assumption behind “on average”, give the worst case, and name three things that can push a real table towards it.

W13-E2 [why] (3 marks) A student's class defines `__hash__` to return 0 for every object. Is a hash table of these objects **wrong**, or only **slow**? Answer for chaining and for open addressing, and give the cost of get and of building a table of n keys.

W13-E3 [how] (4 marks) Compare chaining and open addressing: what a slot holds, the load factors each can tolerate, how each deletes, and one practical advantage of each.

W13-E4 [why] (4 marks) Explain tombstones. Why can an open-addressing delete not simply empty the slot? State how get, put and `_resize` each treat a tombstone, and why the load check must count them.

W13-E5 [why] (3 marks) Show that a hash table that doubles its capacity when the load factor passes a fixed limit does amortised $O(1)$ work per put on rehashing. What happens if it grows by a fixed 8 slots instead?

Part C — Trace the code

W13-T1 [how] Give the index of each key in a table of capacity 8 and in a table of capacity 16: 7, 15, 23, -9, 40, 100. Which keys share a slot at capacity 8 but not at capacity 16?

W13-T2 [how] A `ChainingHashMap(capacity=4)` receives `put(5, 5)`, `put(9, 9)`, `put(13, 13)`, `put(2, 2)` and then `delete(9)`. Show the chains, the size and the load factor after each operation, and say where it resizes.

W13-T3 [how] An `OpenAddressingHashMap()` receives `put` for 3, 11 and 19, then `delete(11)`, `get(19)` and `put(27, 27)`. For each operation list the slots probed, and give the slots at the end.

W13-T4 [how] What does this print?

```
def sum_hash(s):
    return sum(ord(c) for c in s)

def poly_hash(s):
    h = 0
    for c in s:
        h = 31 * h + ord(c)
    return h

for w in ["cat", "act", "tac", "dog", "god"]:
    print(w, sum_hash(w) % 8, poly_hash(w) % 8)
```

(`ord("a")` is 97, and the letters follow in order.) Which pairs collide under each hash, and why is that fair to one of them and not to the other?

W13-T5 [why] An `OpenAddressingHashMap()` receives `put` for 0, 8, 16, 24. Then: `get(24)`, `get(32, None)`, `delete(8)`, `get(24)`, `put(40, 40)`, `get(32, None)`. Give the probes of each operation and the final slots. What do these six operations show about clustering?

Part D — Table state: draw every step

W13-S1 [how] Draw a `ChainingHashMap(capacity=4)` — every bucket, every chain, the size and the capacity — after each of:

`put 10 · put 3 · put 14 · put 7 · put 18 · delete 10 · put 26 · delete 26`

W13-S2 [how] Draw the 8 slots of an `OpenAddressingHashMap()`, with the size and the tombstone count, after each of:

`put 5 · put 13 · put 21 · put 6 · delete 13 · put 29 · delete 5 · put 14`

For each `put`, list the slots probed.

W13-S3 [why] Draw the slots of an `OpenAddressingHashMap()` after each of:

put 1 · put 2 · put 3 · put 4 · put 5 · delete 1 · delete 2 · put 9 · put 10 · put 11

State the load check of each of the last three puts, and whether it causes a sweep at the same capacity, a doubling, or nothing.

Part E — Complexity analysis

W13-K1 [how] a and b are lists of n and m integers. Give Θ on average and in the worst case, and justify:

```
seen = ChainingHashMap()
for x in a:
    seen.put(x, True)
common = 0
for y in b:
    if y in seen:
        common += 1
```

What is it if seen is a Python list and in is the list's in?

W13-K2 [why] n distinct keys are put into a chaining table that **never** resizes, capacity 8. Give the total cost of the n puts. Then give it for a table that doubles at load 0.75, and for one that adds 8 buckets at load 0.75.

W13-K3 [why] Count how many times each word occurs in a text of n words, three ways: (a) for each word, count it by scanning the whole text; (b) sort the words, then count the runs; (c) a hash map from word to count. Give the time of each, the worst case of (c), and which you would use.

Part F — Find and fix the bug

W13-B1 [why] A student's `ChainingHashMap.__contains__`:

```
def __contains__(self, key):
    return self.get(key, None) is not None
```

W13-B2 [how] Part of a `ChainingHashMap._resize`:

```
old = self._buckets
self._buckets = Array(capacity)
for head in old:
    entry = head
    while entry is not None:
        i = self._index(entry.key)
```

```
entry.next = self._buckets[i]
self._buckets[i] = entry
entry = entry.next
```

W13-B3 [why] Part of an `OpenAddressingHashMap.put`, after the load check:

```
for i in self._probe(key):
    slot = self._keys[i]
    if slot is None or slot is TOMBSTONE:
        self._keys[i] = key
        self._values[i] = value
        self._size += 1
        return
    if slot == key:
        self._values[i] = value
        return
```

W13-B4 [how] A `ChainingHashMap.delete`:

```
def delete(self, key):
    i = self._index(key)
    entry = self._buckets[i]
    while entry is not None:
        if entry.key == key:
            self._buckets[i] = entry.next
            self._size -= 1
            return
        entry = entry.next
    raise KeyError(key)
```

Part G — Write the code

In `practice/week13.py`; check with `pytest tests/test_practice_week13.py -v`. Each problem is $O(n)$ with a hash map, and each has a test large enough that an $O(n^2)$ answer is painfully slow. Use your `ChainingHashMap` as the working storage — not a Python dict or set.

W13-C1 [how] `two_sum(values, target)` — two indices whose values add up to the target, in one pass.

W13-C2 [how] `first_repeated(items)` — the first item to appear a second time.

W13-C3 [how] `group_anagrams(words)` — group words that are anagrams of one another, keeping the input order.

W13-C4 [why] `longest_distinct_run(text)` — the length of the longest stretch with no repeated character. Why is it $O(n)$ although the window moves back and forth?

W13-C5 [why] `count_subarrays_with_sum(values, k)` — how many contiguous runs add up to k , with negative values allowed. Why does a sliding window not work here, and why do prefix sums?